Emandoyesus Tesfay
ugr/188057/16

Quality Evaluation of Movie Explorer System

Quality attributes define the non-functional characteristics that determine how well a system meets stakeholder expectations beyond its feature set. This section evaluates the Movie Explorer architecture against three critical quality attributes: **scalability**, **maintainability**, and **performance** and briefly addresses **security** and **reliability**. Each attribute is defined, mapped to specific design decisions in the current stack (React on Vercel, Node.js/Express on Render, PostgreSQL on Neon), and examined for inherent trade-offs and limitations.

Scalability

Scalability is the ability of a system to handle increasing workloads (more users, more requests, or larger datasets) without significant degradation in response time or reliability. A scalable system can grow **horizontally** (adding more instances) or **vertically** (increasing resources per instance), ideally with minimal architectural change.

The Movie Explorer adopts a stateless backend, which is foundational to horizontal scalability. The Express server on Render does not store session data in memory; all authentication state is carried in client-side JWT tokens. Consequently, multiple server instances can run behind Render's load balancer without requiring sticky sessions or shared state. If traffic increases, Render allows manual scaling from a single instance to multiple instances with minimal configuration.

The three-tier separation further supports independent scaling. The **React frontend**, deployed on Vercel, is served as static assets from a global Content Delivery Network. The CDN inherently absorbs traffic spikes (Vercel automatically distributes requests across edge nodes), so the frontend tier scales without any intervention. The **database** tier on Neon separates compute from storage, allowing database CPU and memory to scale independently of disk capacity. Neon's serverless model can automatically scale compute resources up during heavy query periods and down to zero during idle times.

The use of REST APIs contributes to scalability by enforcing a uniform, stateless interaction model. Each request is self-contained, so any **backend** instance can handle any request, making load distribution straightforward.

Trade-Offs and Limitations:

- Horizontal scaling on Render requires a paid plan for multiple instances, introducing a cost that may not be justified for low-traffic periods.
- The database caching layer (using the cached_movies table) reduces external API calls but adds write load; if many users trigger cache misses simultaneously, PostgreSQL could become a bottleneck before the backend tier does.

- Neon’s autoscaling is not instantaneous, there is a brief ramp-up period during which query latency may increase.
- Finally, scaling the TMDB adapter requires careful monitoring because the external API enforces strict rate limits (approximately 40 requests per second on a standard plan). Adding server instances does not increase the TMDB quota, so aggressive caching remains essential.

Maintainability

Maintainability measures how easily a system can be corrected, updated, extended, or understood by developers over time. High maintainability reduces the cost and risk of future changes, whether they are bug fixes, new features, or dependency upgrades.

The architecture is organised into clearly separated layers (controller, service, and data access) within the Express backend. This layering means that a change in one concern, such as switching from one external movie API to another, requires modification only in the TMDB adapter module, without touching controllers or database queries. The React frontend mirrors this modularity through its component-based structure: reusable components like MovieCard and SearchBar can be updated in isolation without affecting page-level logic.

Design pattern usage further improves maintainability. The repository-like grouping of database queries (userQueries, watchlistQueries, movieCacheQueries) centralises SQL in a single location per domain entity. If the database schema evolves, only these files change. The singleton TMDB adapter consolidates API configuration, so credentials and base URLs are not scattered across multiple files.

Using a single programming language (**JavaScript)** across both frontend and backend reduces the cognitive burden on developers, making the codebase more approachable. Standardised REST API responses with a consistent JSON envelope allow frontend developers to write predictable data-handling logic without guessing response shapes.

On the infrastructure side, Git-based continuous integration (**GitHub Actions**) and **automatic deployments** (Vercel and Render connected to the repository) ensure that code quality checks (linting, testing, building) run consistently, catching issues before they reach production.

Trade-Offs and Limitations:

- Modularity introduces a larger number of files and folders, which could initially feel overwhelming to a new developer.
- The strict layering also means that simple operations sometimes require passing data through several layers (controller → service → data access → database), adding boilerplate code.
- Documentation must be maintained alongside the code to explain these patterns, or the layered structure may confuse contributors unfamiliar with the conventions.

Performance

Performance refers to how quickly and efficiently a system responds to user interactions and processes data. Key indicators include response time, throughput, and resource utilization under normal and peak loads.

The Movie Explorer system employs multiple performance-optimising strategies. At the **frontend tier**, Vercel serves the React application as a set of pre-built, minified static files from edge locations close to users. This minimises time-to-first-byte and ensures the interface loads quickly regardless of the user's geographic location. Once loaded, the single-page application only fetches JSON data, avoiding full page reloads and reducing bandwidth consumption.

On the **backend**, Node.js's non-blocking, event-driven runtime is well-suited to I/O-bound workloads. When the server makes a request to the TMDB API or queries the database, it does not block other in-flight requests. This enables a single Render instance to handle many concurrent connections efficiently.

The caching strategy is a central performance feature. The cached_movies table in PostgreSQL stores recently fetched movie metadata with a timestamp. Before calling the external TMDB API, MovieService checks the cache; if data is fresh (e.g., less than 24 hours old), the database query is significantly faster than an external HTTPS round trip. This reduces average response times for popular searches and reduces load on the rate-limited TMDB API.

PostgreSQL's indexing on foreign keys (like user_id and movie_id in watchlist and favourites tables) ensures that queries looking up a user's lists or checking for duplicates execute in milliseconds, even as the tables grow.

Trade-Offs and Limitations:

- Database-based caching is slower than an in-memory cache (such as Redis), which was deliberately avoided to keep the infrastructure simple. This means cached data still incurs a database round trip.

- The cache-invalidation strategy (stale data removed or overwritten) is basic; a user requesting a rarely-accessed, uncached movie will experience the full latency of a TMDB API call.
- During sudden cache-miss storms (e.g., many users searching for a newly released film), the backend could become temporarily slower while populating the cache.
- Finally, Vercel's CDN only accelerates the initial page load; subsequent API response times depend entirely on the backend and database tiers, which are not geographically distributed unless Render's paid regions are used.

Security and Reliability

Security is addressed through **JWT-based authentication**, HTTP header protection via Helmet middleware, and CORS restrictions allowing only the frontend origin.

Passwords are hashed with **bcrypt**. Reliability benefits from Render's automatic restarts on crash, Neon's point-in-time recovery for the database, and the absence of a single point of failure at the frontend tier due to Vercel's distributed CDN.

The primary reliability concern is the dependency on TMDB; if the external API experiences downtime, the Movie Explorer's search and detail features become partially unavailable, though user data in PostgreSQL remains accessible.